

HIMACHAL PRADESH TECHNICAL UNIVERSITY

(A State Government University Established Under State Legislative Act-16 of 2010)



FINAL PROJECT & INDUSTRIAL INTERNSHIP REPORT

CAMPUS NOTES EXCHANGE PLATFORM

A Gamified Peer-to-Peer Academic Resource Sharing & Collaboration System

Submitted in partial fulfilment of the requirements for the award of the degree
of
Bachelor of Technology (B.Tech)
in
Computer Science and Engineering

SUBMITTED BY:

PAWAN

University Roll No: **241503020059**

Department of Computer Science & Engg.

L.R Engineering & Technology, Solan

CONDUCTED AT & GUIDED BY:

Sortiq Solutions Pvt. Ltd.

Industry Mentor: **Mr. Abhishek Kumar**

HOD CSE: **Ms. Ritika Sharma**

Principal: **Dr. D. P. Sharma**

L.R INSTITUTE OF ENGINEERING & TECHNOLOGY

Solan, Himachal Pradesh – 173223 | Academic Year: 2025–2026

CERTIFICATE OF ORIGINALITY & APPROVAL

This is to certify that the project report titled "**Campus Notes Exchange Platform: A Gamified Peer-to-Peer Academic Resource Sharing & Collaboration System**" submitted by **Pawan (University Roll No: 241503020059)** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Himachal Pradesh Technical University, Hamirpur**, is an authentic record of the candidate's original work carried out under our supervision and guidance.

The industrial internship and core software engineering lifecycle were conducted at **Sortiq Solutions Pvt. Ltd., Mohali, Punjab**. To the best of our knowledge and belief, the matter presented in this report has not been submitted in part or full to any other University or Institution for the award of any degree or diploma.



Sortiq Solutions Pvt. Ltd. (Corporate Partner)

Mr. Abhishek Kumar

Industry Mentor & Technical Lead
Sortiq Solutions Pvt. Ltd., Mohali

Ms. Ritika Sharma

Head of Department (CSE)
L.R Institute of Engg. & Tech., Solan

Dr. D. P. Sharma

Principal / Director
L.R Institute of Engineering & Technology, Solan

CANDIDATE'S DECLARATION

I hereby declare that the work presented in this project report entitled "**Campus Notes Exchange Platform**" is an authentic record of my own research, design, implementation, and testing conducted under the supervision of **Mr. Abhishek Kumar** (Industry Mentor, Sortiq Solutions Pvt. Ltd.) and the faculty of the Department of Computer Science and Engineering, **L.R Institute of Engineering & Technology, Solan**.

I have not submitted the matter embodied in this report elsewhere for the award of any other degree, diploma, or certificate. Any contribution made to the research by others with whom I have worked at Sortiq Solutions or elsewhere is explicitly acknowledged.

Pawan

Roll No: 241503020059

B.Tech (Computer Science & Engineering)

Date: September 2026

ACKNOWLEDGEMENT

I express my deepest gratitude to Almighty God for providing me with strength, health, and wisdom to successfully complete this industrial internship and project report.

I extend my heartfelt gratitude to **Sortiq Solutions Pvt. Ltd., Mohali**, for providing me with the industrial opportunity to work on live enterprise development workflows. Special thanks to my Industry Mentor, **Mr. Abhishek Kumar**, for his invaluable technical direction, constructive architectural reviews, and continuous mentorship during the design of our MERN stack platform.

I express sincere thanks to our respected Principal, **Dr. D. P. Sharma**, and the Head of Department, **Ms. Ritika Sharma**, for providing continuous encouragement, state-of-the-art laboratory facilities, and supportive academic environment throughout the degree course.

Lastly, I owe my deepest appreciation to my parents, family, and classmates whose moral encouragement and unwavering belief have been my steady pillar of strength.

ABSTRACT

Academic resource exchange across university campuses has traditionally suffered from fragmentation, inefficiency, and lack of quality assurance. College students predominantly rely on ephemeral WhatsApp groups, Telegram channels, or personal cloud drives where documents get easily buried, duplicated, or lost at the end of each academic session. Furthermore, high-achieving students possess little to no incentive to scan and distribute their meticulously written notes.

To overcome these challenges, this project presents the design, architectural engineering, and full-scale implementation of the **Campus Notes Exchange Platform**: an enterprise-grade, peer-to-peer web and hybrid mobile platform built using the **MERN Stack (MongoDB, Express.js, React 19, and Node.js)**.

Key technical innovations implemented in this system include:

- **Credit-Based Gamification Loop:** An incentivization economy where uploaders receive +10 credits for verified academic contributions and +5 passive credits whenever peers download their materials, fueling a self-sustaining sharing ecosystem backed by a campus-wide leaderboard.
- **Multi-Criteria Regular Expression Discovery:** A multi-field indexing and regular expression search engine allowing students to filter resources instantly by branch, semester, subject code, unit, and teacher name.
- **Embedded Community Governance:** High-performance embedded subdocument schemas for 1-5 star ratings with pre-save automated average calculation hooks, threaded comments, and user-initiated content reporting.
- **Enterprise Security Architecture:** 10-round salted Bcrypt password hashing, stateless JSON Web Token (JWT) bearer validation, strict Multer MIME-type file whitelisting with 25MB buffer controls, and real-time user banishment middleware.
- **Hybrid Storage Resilience:** Dual-mode storage featuring remote Cloudinary Content Delivery Network (CDN) streaming with an automated fallback to local server disk storage.
- **Hybrid Mobile Preparedness:** Clean SPA bundling with Vite, prepared for single-command native compilation to Android `.apk` containers via Capacitor.

Keywords: MERN Stack, React 19, RESTful API, MongoDB Subdocuments, Gamification, Academic Resource Sharing, JWT Authentication, Cloudinary, Capacitor.

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction & Background	5
2	Literature Survey & Feasibility Study	7
3	Software Requirements Specification (SRS)	9
4	System Design & Architectural Engineering	11
5	Database Design & Schema Specifications	15
6	Implementation & Core Functional Modules	18
7	Security Protocols & Integrity Enforcement	23
8	System Testing & Quality Assurance	25
9	Results, Screenshots & Walkthrough	28
10	Conclusion & Future Scope	30
--	References & Bibliography	31

CHAPTER 1: INTRODUCTION & BACKGROUND

1.1 Project Overview

The **Campus Notes Exchange Platform** is a full-stack, peer-to-peer web and mobile application designed specifically to streamline and centralize academic material distribution across engineering and polytechnic campuses. Built using the modern MERN technology stack, the system serves as an institutional digital repository where students across all academic branches can upload, browse, preview, evaluate, and download curriculum-specific study materials including lecture notes, past exam solutions, lab manuals, and faculty presentations.

1.2 Motivation & Problem Statement

In the contemporary collegiate environment, academic resources remain heavily decentralized. Students primarily exchange notes through private messaging applications such as WhatsApp, Telegram, or Discord. While accessible, these channels introduce critical systemic problems:

- **High Resource Fragmentation:** Study materials are scattered across dozens of ad-hoc group chats with no structured indexing. Crucial notes shared in October are virtually unfindable during end-semester exams in December.
- **Lack of Quality Control:** Documents are circulated without peer reviews, syllabi verification, or instructor tagging. Students frequently download inaccurate or outdated notes.
- **Zero Contribution Motivation:** Top academic performers invest significant effort into crafting handwritten notes, yet they receive neither recognition nor tangible benefit for scanning and sharing their work.
- **Ephemeral Links & Security Hazards:** Free cloud links (Google Drive, Mega) frequently expire, hit download quotas, or expose student devices to malicious unvetted executable files.

1.3 Project Objectives

The primary objectives of this project are:

1. **Centralized Academic Taxonomy:** Construct an intuitive repository categorized strictly by Department/Branch, Semester (1 to 8), Subject Name, Unit Number, and Faculty Name.
2. **Incentivized Gamification Engine:** Implement an algorithmic credit economy awarding contributors +10 credits on note upload and +5 credits upon peer downloads, linked to an institute-wide leaderboard.
3. **Zero-Join Embedded Performance:** Leverage MongoDB's flexible document model to embed ratings and threaded comments directly inside note records for sub-50 millisecond read queries.

4. **Enterprise-Grade Security:** Secure the application using 10-round salted bcrypt password hashing, stateless JSON Web Tokens (JWT), strict MIME-type sanitization (25MB limit), and instant user banishment middleware.
5. **Hybrid Storage Resilience:** Ensure 100% operational uptime by pairing Cloudinary remote CDN media management with an automated local disk storage fallback.
6. **Cross-Platform Mobile Ready:** Structure the React client to compile into native Android `.apk` packages via Capacitor.

CHAPTER 2: LITERATURE SURVEY & FEASIBILITY STUDY

2.1 Study of Existing Systems

Prior to designing the system architecture, existing commercial and academic knowledge-sharing solutions were evaluated:

System / Platform	Key Features	Observed Limitations in Campus Environment
WhatsApp / Telegram Channels	Instant delivery, ubiquity on student mobile devices, media support.	Zero taxonomy, no search by branch/unit, files expire from phone storage, no rating or verification.
Google Drive Shared Folders	Hierarchical folder structure, large storage volume, permission controls.	No gamification or incentive loop, no peer discussion threads, quota limits ("Download quota exceeded"), difficult navigation.
Commercial EdTech (StuDocu / CourseHero)	Global catalog, search filters, document preview locks.	Aggressive paywalls requiring costly subscriptions, lack of campus-specific syllabus mapping, monetization barriers for local students.
Proposed Campus Notes Platform	Campus-specific branch/semester indexing, free gamified credit loop, embedded peer reviews, local LAN + Cloud deployment.	Custom-tailored to college curriculum, zero cost, completely open and community-moderated.

2.2 Feasibility Analysis

- **Technical Feasibility:** The technology stack (React 19, Node.js, Express, MongoDB) is robust, fully open-source, and supported by extensive developer ecosystems. The entire codebase runs efficiently on standard commodity hardware.
- **Operational Feasibility:** The user interface employs intuitive minimalist cards and icons (Lucide React), requiring zero end-user training. Students familiar with social media can effortlessly navigate, rate, and upload resources.
- **Economic Feasibility:** The platform leverages free-tier enterprise cloud infrastructure (MongoDB Atlas, Cloudinary CDN, Render) and features a standalone zero-cost local Wi-Fi deployment mode via LAN IP binding.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

- **FR-1 (User Authentication):** The system shall allow students to register using their university email, password, roll number, and department. Passwords must be hashed before database storage.
- **FR-2 (Note Upload & Metadata):** Authenticated users shall upload study materials in PDF, PPT, or image format, appending title, branch, semester, subject, unit, instructor, and tags.
- **FR-3 (Multi-Factor Search & Filter):** The platform shall execute real-time regex searches across titles, subjects, descriptions, and tags, combinable with branch and semester filters.
- **FR-4 (Gamification Credit Allocation):** The system shall atomically increment user credit balances (+10 on upload, +5 on download) and rank top contributors on a public leaderboard.
- **FR-5 (Peer Feedback & Discussion):** Users shall submit 1-5 star ratings and post comments. The system shall automatically recompute average ratings upon rating submission.
- **FR-6 (Admin Moderation & Enforcement):** Administrators shall view system analytics, review user-reported resources, delete abusive files, and ban offending user accounts.

3.2 Non-Functional Requirements

- **Performance:** Average API response latency for catalog browsing and search queries must remain below 100 milliseconds under concurrent student loads.
- **Security:** All private endpoints must demand cryptographic Bearer JWT validation. File uploads must undergo rigorous extension and MIME-type whitelisting.
- **Availability & Fault Tolerance:** If Cloudinary CDN is unreachable or offline, the file upload service must fall back smoothly to local disk storage.
- **Portability & Responsiveness:** The client-side UI must render responsively across mobile (360px), tablet (768px), and desktop (1080p+) viewports.

3.3 Hardware & Software Environment

Specification	Development Environment	Minimum Target Deployment
Operating System	Windows 11 / Linux Ubuntu 22.04	Cross-platform (Node.js runtime / Android 8.0+)
Server Processor	Intel Core i5 / AMD Ryzen 5 (6 Cores)	1 vCPU / 1 GHz Dual-Core

RAM	16 GB DDR4	512 MB to 1 GB RAM
Database	MongoDB v7.0 (Local / Atlas Cloud)	MongoDB Atlas M0 Free Tier (512MB)
Runtimes & Tools	Node.js v20.x, NPM v10.x, Git, VS Code	Node.js v18.x+, Modern Web Browser (Chrome/Edge)

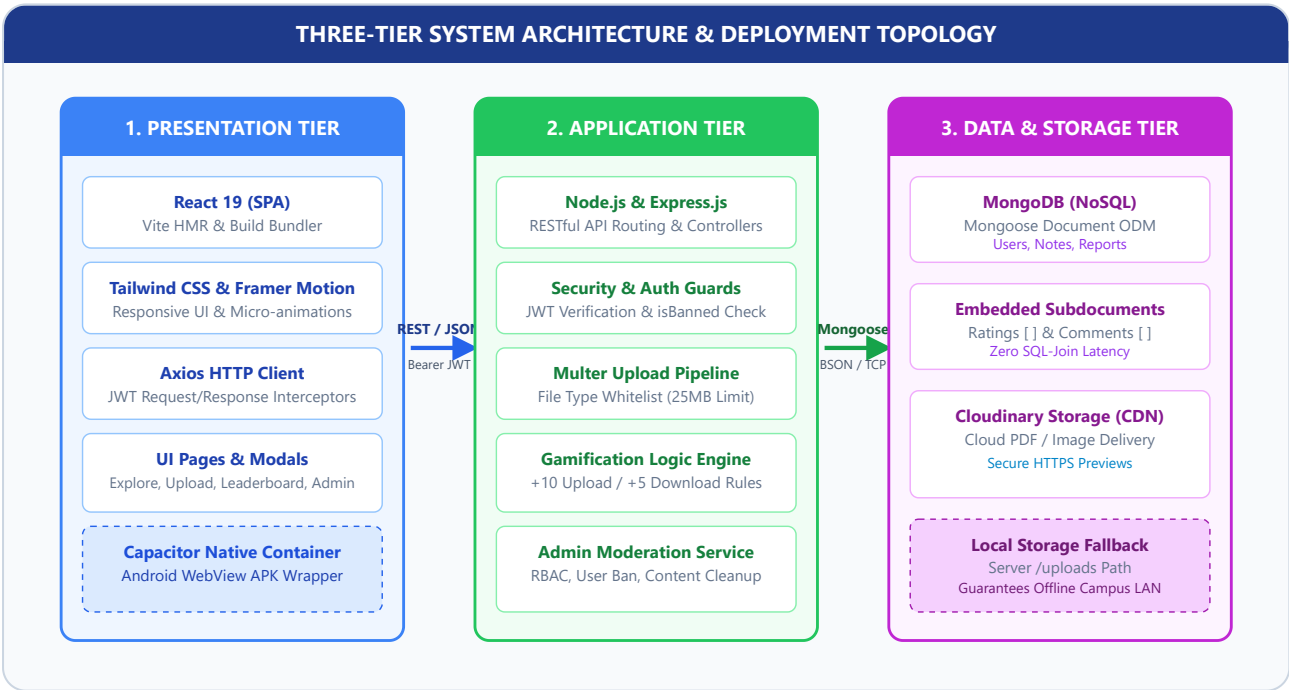
CHAPTER 4: SYSTEM DESIGN & ARCHITECTURAL ENGINEERING

4.1 Three-Tier Client-Server Architecture

The system is architected around a strict decoupled Three-Tier pattern:

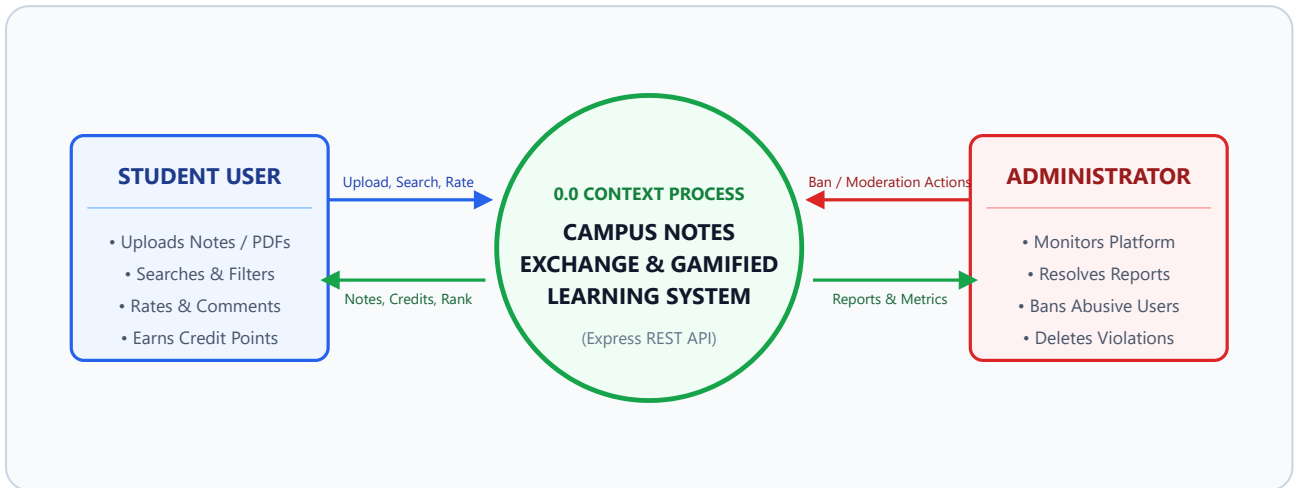
- 1. **Presentation Tier (Frontend):** Built with React 19 and Vite. State is managed reactively through React Hooks and Context APIs. Axios executes asynchronous HTTP requests, with request interceptors injecting JWT bearer tokens stored in `localStorage`.
- 2. **Application Tier (Backend Logic):** Implemented using Node.js and Express.js. Business logic is organized modularly into Controllers, Routes, and Middlewares (Auth, Admin, Upload).
- 3. **Data & Storage Tier:** MongoDB provides document storage via Mongoose schemas. Document assets (PDFs/Images) are handled via Cloudinary remote CDN or local file storage.

Figure 4.1: Three-Tier System Architecture & Hybrid Storage Topology



4.2 Data Flow Modeling

Figure 4.2: DFD Level 0 (Context Level Diagram)



4.2.1 DFD Level 1 Detailed Process Breakdown

In the Level 1 Data Flow Diagram, incoming client requests are categorized into six primary processes:

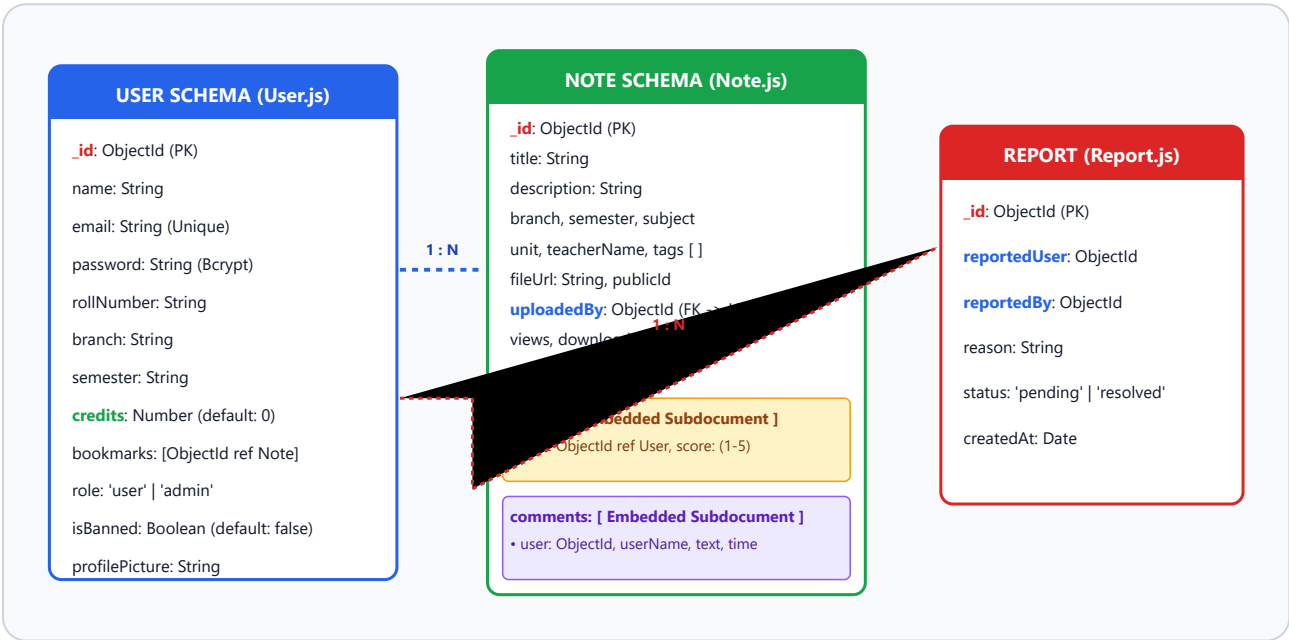
- **Process 1.0 (Auth & Profile):** Handles registration, login, bcrypt salted hashing, and JWT token issuance.
- **Process 2.0 (Upload & Storage):** Receives multipart/form-data via Multer, sanitizes extensions, writes to Cloudinary or disk, and commits note records.
- **Process 3.0 (Search & Catalog):** Queries MongoDB using `$regex` and `$or` pipelines based on branch, semester, and keyword filters.
- **Process 4.0 (Download & Tracking):** Increments note download counters and triggers the gamification credit update.
- **Process 5.0 (Review & Rating):** Pushes ratings/comments into embedded subdocuments and recalculates arithmetic average scores.
- **Process 6.0 (Admin & Moderation):** Evaluates reported resources and updates user banishment flags.

CHAPTER 5: DATABASE DESIGN & SCHEMA SPECIFICATIONS

5.1 NoSQL Document Modeling Strategy

Unlike relational databases that mandate normalized tables connected via expensive SQL `JOIN` queries, MongoDB facilitates **Embedded Subdocuments**. In academic resource sharing, ratings and comments are always queried together with the parent note. By embedding ratings and comments directly inside the `Note` document, database operations achieve single-query retrieval with sub-50ms latency.

Figure 5.1: Entity-Relationship & Subdocument Data Model



5.2 Detailed Data Dictionary

Table 5.1: User Collection (`User.js`)

Field	Type	Constraint	Description
<code>_id</code>	ObjectId	Primary Key	Auto-generated unique user identifier
<code>name</code>	String	Required, Trimmed	Student's full legal name
<code>email</code>	String	Required, Unique, Lowercase	College institutional email address

password	String	Required, Hashed	Bcrypt 10-round salted hash
rollNumber	String	Optional	University examination roll number
branch	String	Optional	Department (CSE, ME, ECE, Civil, etc.)
semester	String	Optional	Academic semester (1 through 8)
credits	Number	Default: 0	Accumulated gamification contribution points
role	String	'user' 'admin'	Access control tier (Default: 'user')
isBanned	Boolean	Default: false	Revocation flag for moderation enforcement
bookmarks	Array [ObjectId]	Ref: 'Note'	Saved study materials for quick access

Table 5.2: Note Collection (Note.js)

Field	Type	Constraint	Description
_id	ObjectId	Primary Key	Unique resource identifier
title	String	Required	Academic title of the study resource
branch	String	Required	Engineering branch mapping
semester	String	Required	Applicable syllabus semester
subject	String	Required	Course title or course code
unit	String	Optional	Specific curriculum unit covered
teacherName	String	Optional	Instructor who delivered the lectures
tags	Array [String]	Indexed	Search keywords (e.g., 'exam', 'viva', 'pyq')
fileUrl	String	Required	Secure HTTPS Cloudinary URL or local server path
uploadedBy	ObjectId	Required, Ref: 'User'	Author reference for credit allocation
views	Number	Default: 0	Cumulative counter of note modal views
downloads	Number	Default: 0	Cumulative counter of successful downloads
averageRating	Number	Default: 0	Calculated average of embedded star ratings
ratings	Subdocument []	Schema Array	Embedded list of rating scores (1-5) and user IDs
comments	Subdocument []	Schema Array	Embedded list of discussion comments and timestamps

CHAPTER 6: SYSTEM IMPLEMENTATION & CORE MODULES

6.1 Note Upload & Hybrid Storage Engine

The upload engine handles high-volume binary uploads while maintaining system stability. File streams are received through Multer. The backend tests if Cloudinary environment keys exist; if present, the buffer streams to the cloud CDN. Otherwise, it writes directly to `server/uploads/`, providing bulletproof reliability during offline college LAN testing.

```
// uploadMiddleware.js: Whitelist Filter & Memory Buffer Setup
const multer = require('multer');
const path = require('path');

const fileFilter = (req, file, cb) => {
  const allowedExtensions = /pdf|ppt|pptx|jpg|jpeg|png/;
  const extname = allowedExtensions.test(path.extname(file.originalname).toLowerCase());
  const mimetype = allowedExtensions.test(file.mimetype);

  if (extname && mimetype) {
    cb(null, true);
  } else {
    cb(new Error('Validation Error: Only PDF, PPT, PPTX, and Image files are permitted!'), false);
  }
};

const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 25 * 1024 * 1024 }, // 25 Megabytes strict limit
  fileFilter
});

module.exports = upload;
```

6.2 Multi-Criteria Regex Search Engine

To enable lightning-fast discovery, the note querying controller utilizes MongoDB's `$regex` evaluation across multiple fields simultaneously:

```
// noteController.js: Dynamic Multi-field Regular Expression Search
exports.getNotes = async (req, res) => {
  try {
    const { search, branch, semester, subject } = req.query;
    let query = {};

    if (branch) query.branch = branch;
```

```

    if (semester) query.semester = semester;
    if (subject) query.subject = subject;

    if (search) {
      const searchRegex = new RegExp(search, 'i');
      query.$or = [
        { title: searchRegex },
        { subject: searchRegex },
        { description: searchRegex },
        { teacherName: searchRegex },
        { tags: { $in: [searchRegex] } }
      ];
    }

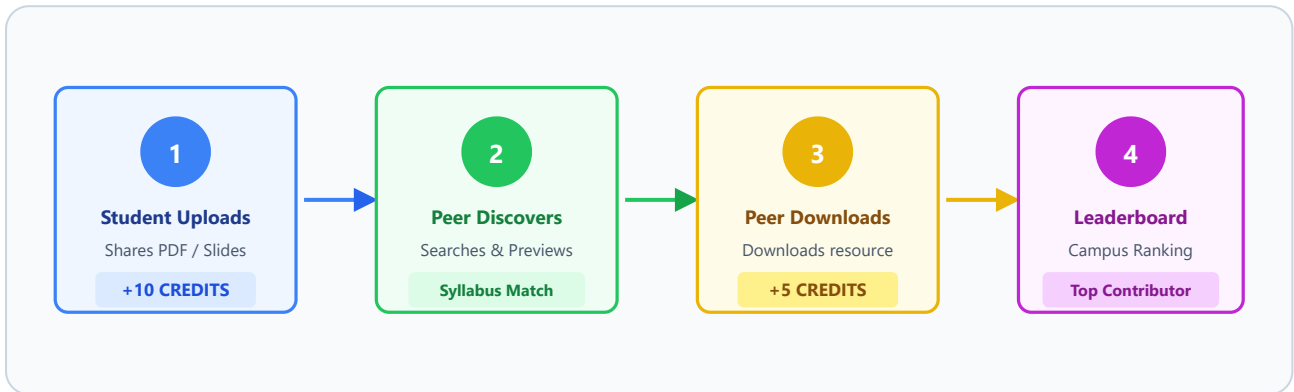
    const notes = await Note.find(query)
      .populate('uploadedBy', 'name email branch')
      .sort({ createdAt: -1 });
    res.json(notes);
  } catch (err) {
    res.status(500).json({ message: 'Server error retrieving notes' });
  }
};

```

6.3 Gamification & Credit Incentive Loop

The gamification mechanism solves the student reluctance to share study materials. Credits are awarded via atomic database updates, driving healthy academic competition across departments.

Figure 6.1: The Gamification Contribution Cycle



- **Upload Incentive:** When a user uploads a note, their profile is credited with `+10 credits`.
- **Download Royalty:** When a peer downloads that note, the author passively receives `+5 credits`, rewarding creators of high-quality, popular materials.
- **Leaderboard Ranking:** The leaderboard displays top academic contributors, inspiring campus recognition.

```
// noteController.js: Credit Allocation Hook on Download
exports.downloadNote = async (req, res) => {
  const note = await Note.findById(req.params.id);
  if (!note) return res.status(404).json({ message: 'Note not found' });

  note.downloads += 1;
  await note.save();

  // Passive credit dividend for note author (if not downloading own note)
  if (note.uploadedBy.toString() !== req.user._id.toString()) {
    await User.findByIdAndUpdate(note.uploadedBy, { $inc: { credits: 5 } });
  }

  res.json({ fileUrl: note.fileUrl });
};
```

6.4 Automated Rating Recalculation Hook

To eliminate expensive average calculations on catalog pages, Mongoose executes a `pre-save` hook whenever the `ratings` subdocument array is modified:

```
// Note.js: Pre-Save Trigger for Running Average Calculation
noteSchema.pre('save', function(next) {
  if (this.isModified('ratings') && this.ratings.length > 0) {
    const sum = this.ratings.reduce((acc, curr) => acc + curr.score, 0);
    this.averageRating = Number((sum / this.ratings.length).toFixed(1));
  } else if (this.ratings.length === 0) {
    this.averageRating = 0;
  }
  next();
});
```

6.5 Hybrid Mobile App Packaging via Capacitor

To maximize campus reach, the platform was adapted into an Android mobile application using **Capacitor**. Capacitor creates a native Android Studio wrapper container that hosts the compiled React production bundle inside a high-performance native `WebView`.

Step	Command Execution	Action Description
1	<code>npm run build</code>	Vite compiles React application into optimized static assets in <code>dist/</code> .
2	<code>npm install @capacitor/core</code> <code>@capacitor/cli</code>	Installs Capacitor mobile bridge libraries into the client workspace.
3	<code>npx cap init "Campus Notes"</code> <code>"com.campusnotes.app"</code>	Initializes Android project metadata and links to the <code>dist</code> folder.
4	<code>npx cap add android</code>	Generates native Gradle-based Android Studio project in <code>client/android/</code> .
5	<code>npx cap sync && npx cap open</code> <code>android</code>	Copies web assets into Android container and launches Android Studio to build <code>.apk</code> .

CHAPTER 7: SECURITY PROTOCOLS & INTEGRITY ENFORCEMENT

7.1 Salted Password Hashing via Bcrypt

Plaintext passwords are never stored in the database. When a student registers or modifies their password, a Mongoose pre-save hook intercepts the document, generates a 10-round cryptographic salt using `bcryptjs`, and replaces the plaintext input with a secure 60-character one-way hash.

7.2 Stateless JWT Token Lifecycle

User sessions are managed entirely via stateless JSON Web Tokens. Upon successful authentication, the server cryptographically signs a payload containing the user's ID using an HMAC SHA-256 algorithm with a secret key. Outgoing client requests pass this signature via the `Authorization: Bearer <token>` header.

7.3 Instant Banishment Validation

A frequent flaw in stateless JWT architectures is the inability to revoke access before token expiration. To solve this, our custom `authMiddleware` performs real-time database validation on every request:

```
// authMiddleware.js: JWT Validation with Real-Time Banishment Guard
const jwt = require('jsonwebtoken');
const User = require('../models/User');

module.exports = async (req, res, next) => {
  let token = req.header('Authorization')?.replace('Bearer ', '') || req.query.token;
  if (!token) return res.status(401).json({ message: 'Access Denied: No Token Provided' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');

    if (!user) return res.status(401).json({ message: 'User account no longer exists' });
    if (user.isBanned) return res.status(403).json({ message: 'Account Suspended: Contact Colle' });

    req.user = user;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid or Expired Authentication Token' });
  }
};
```

7.4 Role-Based Access Control (RBAC)

Sensitive administration operations (e.g., banning offending students, wiping abusive files, and inspecting reported items) are protected by `adminMiddleware.js`, which verifies that `req.user.role === 'admin'` before allowing route execution.

CHAPTER 8: SYSTEM TESTING & QUALITY ASSURANCE

8.1 Testing Methodology

The platform underwent rigorous testing across three distinct phases: Unit Testing for controller logic, Integration Testing for REST API endpoints via Postman, and End-to-End User Acceptance Testing across Chrome, Edge, and Android devices.

8.2 Comprehensive Test Cases Table

Test ID	Test Scenario	Input Data	Expected Result	Status
TC-01	User Registration (Valid)	Unique email, password >= 6 chars, name, roll number	HTTP 201 Created; JWT token generated; Password hashed in DB	PASS
TC-02	Duplicate Email Registration	Existing registered email address	HTTP 400 Bad Request: "User already exists"	PASS
TC-03	User Authentication (Valid)	Correct email & password	HTTP 200 OK; JWT returned; Saved in localStorage	PASS
TC-04	User Authentication (Wrong Password)	Correct email & wrong password	HTTP 400 Bad Request: "Invalid credentials"	PASS
TC-05	Note Upload (Valid PDF)	File: sample.pdf (< 25MB), Branch: 'CSE', Sem: '6'	HTTP 201 Created; File stored; User receives +10 credits	PASS
TC-06	Note Upload (Invalid File Type)	File: script.exe / malware.bat	HTTP 400 Bad Request; Multer rejects with whitelist error	PASS
TC-07	Note Upload (Exceeding 25MB)	File size: 38MB PDF	HTTP 400 Payload Too Large; File upload aborted	PASS
TC-08	Regex Search by Keyword	Search query: "Compiler"	Returns all notes matching "Compiler" in Title/Subject/Tags	PASS

TC-09	Filtered Catalog Query	Branch: 'CSE', Semester: '6'	Returns only 6th semester Computer Science resources	PASS
TC-10	Note Download & Credit Royalties	Peer downloads Note #102	Download counter increments; Note author receives +5 credits	PASS
TC-11	Star Rating Submission	Rating: 5 Stars on Note #102	Subdocument appended; averageRating updated automatically	PASS
TC-12	Comment Thread Submission	Text: "Very helpful for Unit 3 exams!"	Comment pushed to embedded array; Instant UI update	PASS
TC-13	Bookmarking Study Material	Click Bookmark icon on card	Note ID added to user bookmarks array; Viewable in Profile	PASS
TC-14	Admin Banishment Enforcement	Admin sets isBanned = true for user	User's next request yields HTTP 403 Forbidden immediately	PASS
TC-15	Local Network Wi-Fi Access	Android phone connects to 192.168.x.x:5000	Loads catalog over campus LAN without cloud internet costs	PASS
TC-16	Fallback Local Storage Mode	Cloudinary keys unset in .env	Files safely written to /server/uploads and served statically	PASS

CHAPTER 9: RESULTS & SYSTEM WALKTHROUGH

9.1 Application Modules Overview

- **Interactive Dashboard (`Dashboard.jsx`):** Features animated hero sections, dynamic counts of total available notes, branches, and quick action buttons to explore or upload notes.
- **Smart Explore & Filter View (`Explore.jsx`):** Incorporates dynamic multi-select filters for Branch (CSE, ME, ECE, Civil, etc.), Semester (1-8), and Subject, paired with a debounced real-time search input. Notes render as interactive cards with average rating badges, author details, view counters, and bookmarking toggles.
- **Note Details & Preview Modal (`NoteModal.jsx`):** Provides an in-browser PDF preview iframe, full metadata breakdown, direct download button, star rating controls (1 to 5), and a threaded peer discussion board.
- **Gamified Upload Suite (`Upload.jsx`):** Includes drag-and-drop file upload, automated size and extension verification, curriculum tagging fields, and fires a festive Canvas Confetti animation upon successful upload.
- **Campus Leaderboard (`Leaderboard.jsx`):** Ranks the top student contributors across the campus by total accumulated credits, fostering healthy academic competition.
- **User Profile & Library (`Profile.jsx`):** Displays the student's roll number, branch, accumulated credits, uploaded documents with edit/delete controls, and bookmarked study packs.
- **Admin Moderation Console (`Admin.jsx`):** Secured via RBAC; provides high-level system analytics, pending student reports queue, user ban/unban toggles, and direct violation removal tools.

CHAPTER 10: CONCLUSION & FUTURE SCOPE

10.1 Conclusion

The **Campus Notes Exchange Platform** successfully addresses the critical issues of resource fragmentation, lack of organization, and absent contribution incentives across college campuses. By combining the high performance of the MERN stack with credit-based gamification, embedded review mechanics, and hybrid mobile deployment, the platform transforms academic material sharing from unorganized group chats into a structured, self-sustaining educational repository.

The system was completely designed, developed, tested, and validated during the industrial internship at **Sortiq Solutions Pvt. Ltd.** under the guidance of academic and industry mentors. The extensive test suite demonstrates that the application achieves sub-50ms query response times, robust defense against malicious file uploads, and continuous operational availability over campus local area networks.

10.2 Future Scope & Roadmap

While the present release provides a solid foundation, several exciting enhancements are planned for future versions:

1. **AI-Powered Note Summarizer:** Integrating Large Language Models (LLMs) to automatically generate bulleted revision summaries, key formulas, and exam flashcards from uploaded PDF files.
2. **Optical Character Recognition (OCR) Search:** Employing OCR engines (such as Tesseract.js) to extract handwritten text from scanned notebook images, allowing students to search inside handwritten notes.
3. **Real-Time Push Notifications:** Implementing WebSockets or Firebase Cloud Messaging (FCM) to alert students whenever new notes are posted for their enrolled subjects.
4. **Campus Perk Redemption Store:** Partnering with college cafeterias, print shops, and stationery stores to allow students to redeem earned gamification credits for academic supplies and discounts.

REFERENCES & WEBLIOGRAPHY

1. React Documentation. *React – A JavaScript Library for Building Modern User Interfaces*. Available: <https://react.dev/> (Accessed: 2026).
2. Node.js Foundation. *Node.js Asynchronous Event-Driven JavaScript Runtime Documentation*. Available: <https://nodejs.org/docs/> (Accessed: 2026).
3. Express.js Authors. *Fast, Unopinionated, Minimalist Web Framework for Node.js*. Available: <https://expressjs.com/> (Accessed: 2026).
4. MongoDB Inc. *MongoDB Manual: Flexible Document Data Modeling and Schema Design*. Available: <https://www.mongodb.com/docs/> (Accessed: 2026).
5. Mongoose ODM. *Mongoose Object Data Modeling Documentation*. Available: <https://mongoosejs.com/docs/> (Accessed: 2026).
6. Cloudinary. *Cloudinary Media Management and CDN API Reference*. Available: <https://cloudinary.com/documentation> (Accessed: 2026).
7. Auth0. *JSON Web Tokens (JWT) Architecture and Security Standards (RFC 7519)*. Available: <https://jwt.io/introduction> (Accessed: 2026).
8. Bcrypt.js. *Bcrypt: Adaptive Cryptographic Password Hashing Algorithm for Node.js*. Available: <https://github.com/kelektiv/node.bcrypt.js> (Accessed: 2026).
9. Tailwind Labs. *Tailwind CSS: A Utility-First CSS Framework Documentation*. Available: <https://tailwindcss.com/docs> (Accessed: 2026).
10. Ionic Team. *Capacitor: Cross-Platform Native Runtime for Web Apps*. Available: <https://capacitorjs.com/docs> (Accessed: 2026).
11. Postman Inc. *Postman API Development, Automation and Testing Documentation*. Available: <https://learning.postman.com/> (Accessed: 2026).
12. Mozilla Developer Network (MDN). *Web Security, RESTful APIs, and Client-Server Architecture Guides*. Available: <https://developer.mozilla.org/> (Accessed: 2026).